



An Internship Report on

ML & Edge Computing enabled EMS for optimization of Hybrid Electric Powertrain

Submitted partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

in

Computer Science and Engineering (Artificial Intelligence and Machine Learning)

By

Mishael Julian (2462184)

Under the Guidance of

Dr. Sujatha A K

Department of AI and Data Science Engineering

School of Engineering and Technology,

CHRIST (Deemed to be University),

Kumbalagodu, Bengaluru - 560 074

September 2026



CHRIST
(DEEMED TO BE UNIVERSITY)
BANGALORE | DELHI NCR | PUNE

Vision

“Excellence and Service”

Mission

CHRIST (Deemed to be University) is a nurturing ground for an individual's holistic development to make effective contribution to the society in a dynamic environment.

Core Values

*Faith in God
Moral Uprightness
Love of Fellow Beings
Social Responsibility
Pursuit of Excellence*



CHRIST
(DEEMED TO BE UNIVERSITY)
BANGALORE | DELHI NCR | PUNE

Department Vision

“To excel in human-centred AI and data-driven innovation through ethical research, societal well-being, and transformative collaborations”

Department Mission

- M1:** *Empowering individuals to ethically harness data and AI through accessible and value-driven curriculum.*
- M2:** *Department Foster a dynamic research environment that advances innovative and impactful solutions for the betterment of global well-being.*
- M3:** *Innovate scientific knowledge and Entrepreneurship through academia and Industry collaborations.*

Program Educational Objectives (PEOs)

- PEO1:** *Professional Acumen: Understand, analyze and design solutions with professional competency for the real-world problems.*
- PEO2:** *Critical Analysis: Develop software/embedded solutions for the requirements, based on critical analysis and research.*
- PEO3:** *Team work: Function effectively in a team and as an individual in a multidisciplinary/multicultural environment*
- PEO4:** *Life Long Learning: Accomplish holistic development comprehending professional responsibilities.*

Program Outcomes (POs)

PO1: Engineering Knowledge *Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization to develop the solution of complex engineering problems.*

PO2: Problem Analysis *Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.*

PO3: Design/Development of Solutions *Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.*

PO4: Conduct Investigations of Complex Problems *Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions.*

PO5: Engineering Tool Usage *Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.*

PO6: The Engineer and The World *Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.*

PO7: Ethics *Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.*

PO8: Individual and Collaborative Team work *Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.*

PO9: Communication *Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.*

PO10: Project Management and Finance *Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.*

PO11: Life-Long Learning *Recognize the need for, and have the preparation and ability for (i) independent and life-long learning (ii) adaptability to new and emerging technologies and (iii) critical thinking in the broadest context of technological change.*

Program Specific Outcomes (PSOs)

PSO1: Artificial Intelligence *Utilize artificial intelligence principles to solve real-world engineering problems.* Applied through the design of a **Predictive Energy Management System (EMS)** for a Hybrid Electric Vehicle, which combines machine learning, predictive control, and intelligent decision-making to optimize power distribution, battery utilization, and vehicle energy efficiency under varying driving conditions.

PSO2: Machine Learning *Select suitable machine learning algorithms for the given problem.* Applied through the development of **Random Forest-based predictive models** for forecasting vehicle power demand, speed, and acceleration, enabling proactive energy management decisions while maintaining high prediction accuracy and computational efficiency suitable for real-time deployment.

PSO3: Sustainable Engineering and Service Learning *Design intelligent engineering solutions that address real-world societal challenges.* Applied through the development of an intelligent Hybrid Electric Vehicle Energy Management framework that improves fuel efficiency, reduces battery degradation, lowers emissions, and promotes sustainable transportation using AI-assisted supervisory control.



CHRIST
(DEEMED TO BE UNIVERSITY)
BANGALORE | DELHI NCR | PUNE

School of Engineering and Technology

Department of AI and Data Science Engineering

CERTIFICATE

This is to certify that **Mishael Julian (2462184)** has successfully completed his/her internship work entitled “**ML & Edge Computing enabled EMS for optimization of Hybrid Electric Powertrain**” at **CHRIST University, Mysore Rd, Kanmanike, Kengeri, Kumbalagodu, Bengaluru, Karnataka 560074** , from **4th May** to **6th June** for a duration of **32** days and submitted on «Internship Viva Date» in partial fulfillment for the award of **Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning)** during the academic year **2026–27**.

GUIDE

Dr. Sujatha A K
(Signature with Date)

HEAD OF THE DEPARTMENT

Dr. Michael Moses T
(Signature with Seal)

EXAMINER 1

(Name & Signature with Date)

Examiner 2

(Name & Signature with Date)

ACKNOWLEDGEMENT

I would like to thank **Dr Rev Fr Joseph C. C.**, Vice Chancellor, CHRIST (Deemed to be University), **Dr Rev Fr Viju P. D.**, Pro Vice Chancellor, **Dr Fr Jiby Jose**, Director, School of Engineering and Technology, **Fr Shijin P. J.**, Assistant Director, School of Engineering and Technology, CHRIST (Deemed to be University), **Dr Raghunandan Kumar R.**, Dean, and **Dr Mary Anitha E. A.**, Associate Dean, for their kind patronage.

I would also like to express sincere gratitude and appreciation to **Dr. Michael Moses T** , Head of the Department, **Department of AI and Data Science Engineering** for giving me this opportunity to take up this project.

I am also extremely grateful to my guide, **Dr. Sujatha A K**, who has supported and helped to carry out the project. Her constant monitoring and encouragement helped me keep up to the project schedule.

I am also extremely grateful to my co-guide, **Mr. Martin Dsouza** , who has supported and helped to carry out the project. His constant monitoring and encouragement helped me keep up to the project schedule.

DECLARATION

I hereby declare that the project titled “**ML & Edge Computing enabled EMS for optimization of Hybrid Electric Powertrain**” is a record of the original internship work undertaken as part of the course **CSE581 - INTERNSHIP 1** , in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology**.

I completed this internship under the supervision of **Dr. Sujatha A K** and **Mr. Martin Dsouza** , at **CHRIST University , Mysore Rd, Kanmanike, Kengeri, Kumbalagodu, Bengaluru, Karnataka 560074** , during the period from **4th May** to **6th June** .

I further declare that this internship report has not been submitted, either in whole or in part, for the award of any degree, diploma, associateship, fellowship, or any other academic qualification at any other institution. Furthermore, this report has not been submitted for publication or presentation elsewhere.

I affirm that this internship work has been carried out in accordance with the Code of Research Conduct prescribed by the University.

Place: CHRIST (Deemed to be University), Kumbalagodu, Bengaluru - 560 074

Date: 4th September 2026

Signature: Mishaal Julian (2462184)

Contents

1	Introduction	1
1.1	Objectives	1
1.2	Company / Project Profile	2
2	Technical Description and Implementation	3
2.1	Actual Work	3
2.1.1	Prerequisites	3
2.1.2	Responsibilities	3
2.1.3	Challenges	4
2.1.4	EMS Performance Summary	4
2.1.5	Vehicle Power Demand	5
2.1.6	Rule-Based Supervisory EMS Algorithm	5
2.2	System Architecture and Data Flow	7
2.3	Telemetry Preprocessing and Feature Engineering	8
2.4	Predictive Demand Forecasting	9
2.5	SOC Planning and Supervisory Mode Control	10
2.6	Simulation, Validation, and Implementation Outcomes	11
3	Conclusion and Future Scope	12
3.1	Conclusion	12
3.2	Future Scope	12
	Appendices	14
A.1	Offer Letter	14
A.2	Completion Certificate	15
A.3	Code and Screenshots	16
A.3.1	Loading and Preparing Telemetry Data	16
A.3.2	Demand Forecasting Module	19
A.3.3	Hybrid Decision Engine	22
	References	25

LIST OF FIGURES

2.1	Project PHEX '26 Supervisory Energy Management System Architecture	4
A.3.1	Driver Behaviour Dataset Preprocessing Pipeline	16
A.3.2	IMU Telemetry Feature Engineering	17
A.3.3	NASA Battery Dataset Processing and Feature Extraction	17
A.3.4	EMS Feature Engineering and State Variable Generation	18
A.3.5	Demand Forecasting Module Initialization and Model Loading	19
A.3.6	Future Vehicle Demand Prediction Algorithm	20
A.3.7	Random Forest Demand Forecaster Training Pipeline	21
A.3.8	Hybrid Decision Engine Initialization and Model Loading	22
A.3.9	Machine Learning-Based EMS Decision-Making Workflow	23
A.3.10	Supervisory Rule-Based Energy Management Decision Logic	24

LIST OF TABLES

2.1	EMS Performance Summary	4
2.2	Representative Feature Groups Used by the Predictive EMS	8
2.3	Supervisory EMS Operating Modes	10

CHAPTER 1

INTRODUCTION

The automotive industry is undergoing a rapid transformation toward sustainable and intelligent transportation systems. Hybrid Electric Vehicles (HEVs) have emerged as an effective solution for reducing fuel consumption and greenhouse gas emissions while maintaining the driving range and reliability of conventional internal combustion engine vehicles. A key component of every HEV is the Energy Management System (EMS), which determines how power is distributed between the internal combustion engine, electric motor, and battery under varying driving conditions [1–4].

Traditional rule-based Energy Management Systems rely on predefined control strategies that are unable to anticipate future traffic conditions, route characteristics, or driver demand [1]. Recent advances in Artificial Intelligence (AI) and Machine Learning (ML), including tree-based ensemble methods, provide opportunities to develop predictive control systems capable of improving fuel economy, battery utilization, regenerative braking performance, and overall vehicle efficiency [5, 6].

This internship was carried out as part of Project PHEX '26 (Predictive Hybrid Energy Executive), an academic research initiative focused on developing an AI-assisted supervisory Energy Management System for Hybrid Electric Vehicles. The work involved designing and implementing predictive control modules, traffic-aware decision-making, demand forecasting algorithms, State-of-Charge (SOC) planning strategies, rule-based supervisory control, and simulation-based performance evaluation using Python and Scikit-learn [7]. The developed PoC demonstrates how predictive intelligence can be integrated with conventional EMS strategies to improve overall hybrid vehicle operation while maintaining system safety and reliability.

1.1 Objectives

The primary objectives of the internship were:

- Study the architecture and operation of Hybrid Electric Vehicle Energy Management Systems.
- Design a supervisory rule-based Energy Management System for intelligent power management.
- Develop predictive demand forecasting models using Machine Learning techniques.
- Implement traffic-aware route prediction for proactive energy management.
- Design a State-of-Charge (SOC) planning strategy for battery optimization.

- Develop an intelligent decision engine capable of selecting optimal operating modes.
- Integrate safety override mechanisms to ensure reliable EMS operation.
- Validate the proposed EMS through simulation and performance evaluation using multiple Key Performance Indicators (KPIs).

1.2 Company / Project Profile

Project PHEX '26 (Predictive Hybrid Energy Executive) is a student-led research and development initiative undertaken under the guidance of faculty mentors from CHRIST (Deemed to be University). The project focuses on developing an intelligent Energy Management System for Hybrid Electric Vehicles by integrating Artificial Intelligence, Machine Learning, predictive control algorithms, and vehicle simulation technologies.

The objective of Project PHEX '26 is to build a scalable PoC capable of predicting future driving conditions and optimizing energy distribution between the engine, electric motor, and battery. The project combines traditional rule-based supervisory control with predictive decision-making models to improve fuel economy, battery utilization, regenerative braking efficiency, and overall vehicle performance. During this internship, the primary contribution involved the development of predictive EMS modules, including demand forecasting, route-aware SOC planning, intelligent mode selection, safety override logic, simulation workflow, performance benchmarking, and technical documentation.

CHAPTER 2

TECHNICAL DESCRIPTION AND IMPLEMENTATION

2.1 Actual Work

During the internship, the primary responsibility was the development of an AI-assisted supervisory Energy Management System (EMS) for Hybrid Electric Vehicles under Project PHEX '26. The implementation combined conventional rule-based control with predictive machine learning techniques to improve battery utilization, fuel economy, and overall vehicle efficiency. The work involved designing predictive control modules, implementing simulation models, integrating subsystem communication, and validating the complete EMS through multiple drive-cycle simulations.

2.1.1 Prerequisites

- Strong programming knowledge in Python.
- Understanding of Hybrid Electric Vehicle (HEV) architecture.
- Knowledge of Energy Management Systems (EMS).
- Familiarity with Machine Learning concepts and Scikit-learn.
- Understanding of NumPy, Pandas, and Matplotlib.
- Knowledge of predictive modelling and Random Forest regression.
- Basic understanding of vehicle dynamics and battery management.
- Familiarity with Git-based software development and modular Python programming.

Figure 2.1 presents the complete supervisory EMS architecture used in Project PHEX '26. It shows how route prediction, demand forecasting, SOC planning, safety validation, mode selection, and torque control interact with the hybrid powertrain components.

2.1.2 Responsibilities

- Task completion within deadlines
- Daily reporting to mentor
- Independent problem-solving

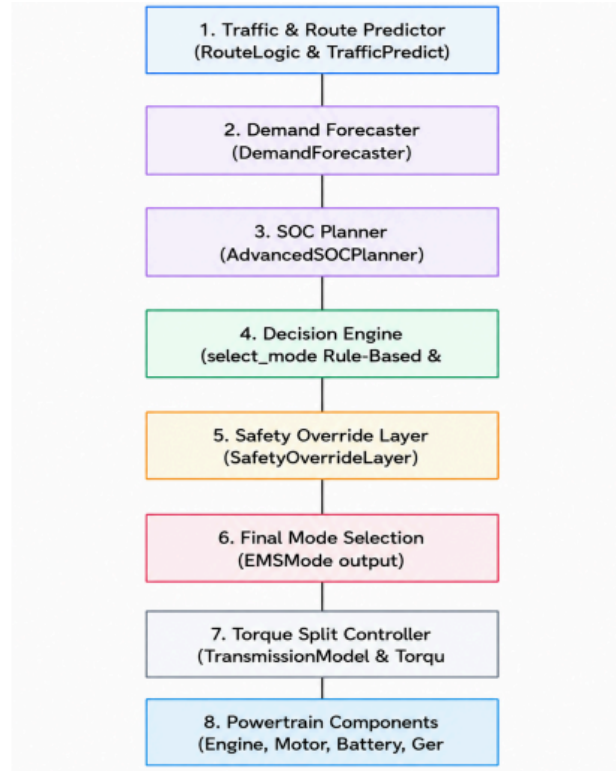


Figure 2.1: Project PHEX '26 Supervisory Energy Management System Architecture

2.1.3 Challenges

Dealing with missing values, outliers, inconsistencies, and feature selection were major challenges. Imputation strategies required careful consideration.

2.1.4 EMS Performance Summary

Table 2.1 summarizes the principal KPIs obtained from the EMS simulation. These values provide a concise view of fuel use, SOC regulation, regenerative-energy recovery, operating-mode distribution, efficiency, emissions, cost, and battery-power demand.

Performance Metric	Value	Unit
Fuel Economy	1.27	L/100 km
SOC Deviation (RMS)	1.86	%
Energy Recuperation	92.0	%
EV Mode Share	39.9	%
Hybrid Mode Share	13.8	%
ICE Mode Share	17.3	%
Operating Cost	0.0358	Currency/km
CO ₂ Emissions	29.3	g/km
Average ICE Efficiency	33.2	%
Peak Battery Power	22.17	kW

2.1.5 Vehicle Power Demand

The Demand Forecaster estimates the power required to move the vehicle under changing road and driving conditions. Following standard electric and hybrid vehicle modelling principles [3,4], the total traction force is expressed as

$$F_{\text{traction}} = F_{\text{rolling}} + F_{\text{aero}} + F_{\text{grade}} + F_{\text{acceleration}}, \quad (2.1)$$

and the corresponding vehicle power demand is

$$P_{\text{demand}} = F_{\text{traction}} v, \quad (2.2)$$

where P_{demand} is the required vehicle power in watts, F_{traction} is the total traction force in newtons, and v is the vehicle speed in metres per second. The traction force comprises rolling resistance, aerodynamic drag, road-grade resistance, and acceleration force.

2.1.6 Rule-Based Supervisory EMS Algorithm

The implemented `select_mode()` function applies rule-based supervisory control to select EV, Hybrid, or ICE operation while accounting for vehicle demand, battery state, traffic conditions, route context, and safety constraints.

Algorithm 1 Rule-Based Supervisory EMS Mode Selection Algorithm

```
def select_mode(
    telemetry: dict,
    traffic_state: TrafficState,
    soc_plan: SOCPlan,
    battery_health: BatteryHealthState,
    predicted_demand_kw: float,
    regen_result: RegenResult,
) -> EMSMode:
    speed = telemetry.get("speed_kmh", 0.0)
    soc = telemetry.get("soc", 0.0)
    soc_frac = soc / 100.0 if soc > 1.0 else soc
    power_demand = telemetry.get("power_demand_kw", 0.0)
    throttle = telemetry.get("throttle", 0.0)
    current_segment = telemetry.get("current_segment", "urban")
    next_segment = telemetry.get("next_segment", "urban")
    position = telemetry.get("position_kmh", 0.0)
    rpm = telemetry.get("rpm", 0.0)

    # 1. Regenerative Braking
    if power_demand < 0.0:
        if soc_frac >= MAX_SOC:
            logger.info("RULE6_REGEN: Regen disabled - batte (attribute) __mul__: float ;")
        else:
            clamped_regen = min(abs(power_demand), MOTOR_PEA
            # 1 kWh = 1/3600 kWh
            recovered_wh = clamped_regen * (1.0 / 3600.0) * 1000.0
            logger.info("RULE6_REGEN: Regenerative braking active. Recovered Wh: %.4f", recovered_wh)
            return EMSMode.REGEN

    # 2. Maximum Acceleration
    if throttle >= 0.90:
        logger.info("RULE7_MAX_ACCEL: Maximum acceleration requested.")
        return EMSMode.HYBRID_ASSIST

    # 3. Low SOC Recovery
    if soc_frac <= 0.25:
        logger.info("RULE8_SOC_RECOVERY: Low SOC (SOC=%.2f) recovery active.", soc_frac)
        return EMSMode.CHARGE_SUSTAIN

    # 4. Urban Stop-and-Go EV Priority
    if speed < 50.0 and (traffic_state == TrafficState.CONGESTED or traffic_state.value == "CONGESTED") and current_segment.lower() == "urban":
        if soc_frac <= (MIN_SOC + 0.05):
            logger.info("RULE1_SUPPRESSED_LOW_SOC: Urban EV Priority suppressed due to low SOC (SOC=%.2f)", soc_frac)
        else:
            logger.info("RULE1_EV_PRIORITY: Urban stop-and-go prioritising EV operation.")
            return EMSMode.EV

    # 5. SOC Preservation Before Urban Zones
    upcoming_urban = next_segment.lower() in ("urban", "stop_go")
    if upcoming_urban and soc_frac < 0.80 and current_segment.lower() in ("highway", "mixed", "arterial"):
        logger.info("RULE2_SOC_PRESERVE: Upcoming urban zone - preserving SOC (SOC=%.2f).", soc_frac)
        return EMSMode.CHARGE_SUSTAIN

    # 6. Traffic-Aware Mode Planning
    if traffic_state == TrafficState.CONGESTED or traffic_state.value == "CONGESTED":
        logger.info("RULE3_TRAFFIC_AWARE: High congestion forecast - biasing EV/Hybrid mode.")
        return EMSMode.HYBRID_ASSIST

    # 7. GPS Route Preview Mode Planning
    if current_segment.lower() == "urban":
        logger.info("RULE4_ROUTE_PREVIEW: Urban segment - favouring EV mode.")
        return EMSMode.EV
    elif current_segment.lower() == "highway":
        logger.info("RULE4_ROUTE_PREVIEW: Highway segment - favouring ICE efficiency mode.")
        return EMSMode.ICE

    # 8. ICE Efficiency Sweet Spot (C.RULE5)
    if rpm > 0.0 and (rpm < 2000.0 or rpm > 3000.0):
        logger.info("RULE5_RPM_SWEET_SPOT: Engine outside peak band (%d RPM). Adjusting split.", int(rpm))
        if rpm > 3000.0:
            # Shed load
            return EMSMode.HYBRID_ASSIST
        else:
            # Deactivate or increase load
            return EMSMode.EV
```

2.2 System Architecture and Data Flow

The Project PHEX '26 Energy Management System was organized as a modular supervisory-control pipeline. This structure separates prediction, planning, decision-making, safety validation, and actuator-command generation so that each module can be developed and tested independently. As illustrated in Figure 2.1, raw telemetry first enters the predictive layer and is progressively transformed into an operating-mode decision and a power-split command.

Representative component limits and powertrain characteristics can be parameterized using manufacturer technical data, such as the Mercedes-Benz A250e plug-in hybrid specifications [8]. The principal information flow is as follows:

1. The traffic and route predictor identifies the current road context and estimates the likely characteristics of the upcoming segment.
2. The demand forecaster uses recent telemetry history to predict future power demand, speed, and acceleration.
3. The SOC planner interprets the predicted route and demand information to determine whether battery energy should be preserved, consumed, or replenished.
4. The decision engine combines the predictive advice with deterministic vehicle-operating rules.
5. The safety override layer checks the proposed decision against battery, powertrain, and operating constraints.
6. The final mode-selection block produces the validated EMS operating mode.
7. The torque-split controller converts the selected mode into engine and motor torque requests.
8. The powertrain model applies these requests to the engine, motor, battery, generator, and transmission subsystems.

Communication between modules was implemented through clearly defined Python data structures. Telemetry values such as vehicle speed, acceleration, requested power, battery SOC, battery temperature, braking status, road grade, traffic state, and route segment were grouped into a common vehicle-state representation. This reduced direct dependencies between modules and allowed simulation components to be replaced without redesigning the complete controller. The modular interface also provides a migration path toward standardized in-vehicle communication based on SAE J1939 and CAN FD [9, 10].

The modular design supports graceful degradation: unavailable, incomplete, or unsafe predictions trigger deterministic rule-based control, preserving safe and predictable operation while retaining the benefits of predictive advice.

2.3 Telemetry Preprocessing and Feature Engineering

The predictive EMS depends on consistent data collected from sources with different sampling rates, units, and data quality. A common preprocessing pipeline was therefore developed for driver-behaviour records, IMU measurements, battery ageing data, route information, and vehicle telemetry. The pipeline removed duplicate samples, handled missing observations, clipped extreme statistical outliers, encoded categorical fields, and standardized numerical features before model training.

Four groups of derived information were particularly important:

- **Driver-behaviour features:** throttle demand, braking intensity, steering activity, lane deviation, reaction time, and synthetic efficiency indicators.
- **IMU features:** resultant acceleration magnitude, gyroscope magnitude, axis-wise jerk, and total jerk magnitude for identifying rapid vehicle manoeuvres.
- **Battery features:** voltage, current, temperature, elapsed time, discharge duration, and a capacity-related degradation proxy obtained from NASA battery-cycle data.
- **EMS state features:** high-power demand, low or critical SOC, urban/highway context, regenerative-braking eligibility, road grade, thermal stress, and predicted next-segment conditions.

Table 2.2 organizes the main input variables according to their source and function within the predictive controller. It highlights how vehicle, battery, route, traffic, and temporal information contribute to demand estimation and supervisory decision-making.

Table 2.2: Representative Feature Groups Used by the Predictive EMS

Feature Group	Representative Variables	Purpose
Vehicle state	Speed, acceleration, braking, throttle	Estimate immediate traction demand
Battery state	SOC, temperature, voltage, current	Protect battery limits and plan energy use
Route context	Segment type, grade, next segment	Prepare for upcoming road conditions
Traffic context	Urban, mixed, congested, stop-and-go	Bias EV/Hybrid mode selection
Temporal history	Lagged speed, acceleration, power	Forecast near-future vehicle demand

Consistent feature preparation during training and runtime prevented input mismatch, while trip-level grouping reduced temporal leakage. The processed data supplied a unified interface for the demand forecaster, SOC planner, Hybrid Decision Engine, and KPI evaluation workflow.

2.4 Predictive Demand Forecasting

The Demand Forecaster was designed to provide short-horizon estimates of vehicle power demand, speed, and acceleration. Instead of making a decision using only the current sample, the module constructs rolling-window features from the recent telemetry history. Five lag values for speed, acceleration, and required power are combined with battery, route, traffic, braking, and regenerative-availability information to represent the temporal driving context.

Three Random Forest regression models are trained independently for the forecast variables using the ensemble-learning approach introduced by Breiman [6]. For a forecast horizon of h samples, the targets are defined as

$$y_t^{(P)} = P_{t+h}, \quad y_t^{(v)} = v_{t+h}, \quad y_t^{(a)} = a_{t+h}, \quad (2.3)$$

where P , v , and a denote required power, vehicle speed, and acceleration. Independent regressors allow each output to learn a suitable relationship while retaining a simple runtime interface.

Model evaluation uses trip-level train/test partitioning to reduce temporal leakage. Prediction quality is measured using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2):

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, \quad (2.4)$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}, \quad (2.5)$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}. \quad (2.6)$$

During simulation, the trained models are loaded dynamically from local storage. The runtime feature vector is aligned with the feature order retained during training, categorical route and traffic values are converted to numerical representations, and missing optional values are assigned safe defaults. If the models cannot be loaded, the module returns the current measured state as a fallback rather than interrupting EMS execution.

The predictions do not directly control the powertrain. They are supplied to the supervisory layer as advisory inputs that indicate whether demand is likely to rise or fall. This separation enables proactive battery-energy planning while ensuring that final commands remain subject to deterministic rules and safety validation.

2.5 SOC Planning and Supervisory Mode Control

The State-of-Charge planner uses route context, traffic conditions, and predicted power demand to determine how battery energy should be used over the upcoming driving segment. Urban and stop-and-go conditions generally provide opportunities for electric propulsion and regenerative braking, whereas a predicted highway segment or sustained high demand may justify preserving SOC or operating the engine near an efficient region.

The supervisory controller converts this planning advice into a discrete operating mode. Table 2.3 summarizes the principal modes represented in the PoC, together with their typical activation context and control objective. It clarifies how the EMS balances energy recovery, electric driving, hybrid assistance, engine operation, and SOC preservation.

Table 2.3: Supervisory EMS Operating Modes

Operating Mode	Typical Selection Context	Supervisory Objective
Regenerative braking	Negative power demand with available battery capacity	Recover kinetic energy
EV mode	Low-speed urban operation with adequate SOC	Reduce fuel use and local emissions
Hybrid assist	High acceleration or congested operation	Share demand between engine and motor
ICE mode	Highway operation or efficient engine-speed region	Use the engine efficiently
Charge sustain/recovery	Low SOC or an upcoming high-demand segment	Restore or preserve battery energy

The rule hierarchy prioritizes vehicle safety and battery protection over optimization. Regenerative braking is evaluated first because it is associated with negative traction demand. Maximum-acceleration requests and low-SOC recovery are then handled before route-aware EV preference, traffic-aware hybrid assistance, highway preview, and engine-efficiency rules. A deterministic default mode is returned when none of the specialized conditions is active.

Before a mode is accepted, the Safety Override Layer verifies that it is compatible with the current vehicle state. Relevant checks include critical SOC, excessive battery temperature, infeasible charging or discharging requests, regenerative braking availability, and requests that would exceed component operating constraints. Unsafe ML recommendations are rejected and replaced by the rule-engine result. This arrangement prevents prediction uncertainty from bypassing essential protection logic.

The validated mode is subsequently interpreted by the torque-split controller. EV operation assigns the demand primarily to the motor, ICE operation emphasizes engine torque, and Hybrid mode distributes the request between both sources. Regenerative operation commands negative motor torque within battery acceptance and motor-power limits.

2.6 Simulation, Validation, and Implementation Outcomes

The integrated EMS was evaluated through a simulation-driven workflow before any consideration of embedded deployment. Representative WLTP Urban and WLTP Mixed profiles, based on the Worldwide Harmonized Light Vehicles Test Procedure [11], together with a Bangalore Urban driving profile, were used to exercise low-speed traffic, repeated acceleration and braking, mixed operating conditions, and sustained demand. At every simulation step, the framework updated the vehicle state, generated predictive advice, selected and validated an operating mode, allocated torque, updated battery SOC, and recorded performance variables.

The validation workflow consisted of the following stages:

1. Load and preprocess the selected drive-cycle and route context.
2. Initialize the vehicle, powertrain, battery, prediction, and control modules.
3. Execute the supervisory EMS sequentially for each telemetry sample.
4. Record fuel use, SOC trajectory, engine runtime, recovered energy, mode selection, component power, and operating efficiency.
5. Aggregate the recorded signals into the KPIs reported in Table 2.1.

The KPI summary indicates that the controller maintained an SOC RMS deviation of 1.86% while recovering 92.0% of the available regenerative energy represented by the simulation. The mode distribution shows that EV operation accounted for 39.9% of the evaluated samples, with Hybrid and ICE operation contributing 13.8% and 17.3%, respectively. The reported average ICE efficiency of 33.2% and peak battery power of 22.17 kW provide additional evidence that both energy sources were actively managed within the simulated operating envelope.

Implementation required careful attention to missing telemetry, categorical encoding, temporal feature alignment, outlier handling, model availability, and coordination between predictive and deterministic modules. These issues were addressed through reusable preprocessing functions, explicit state objects, modular Python classes, logged fallback behaviour, and safety-first rule ordering. Version-controlled development also supported incremental testing of individual modules before integration.

The resulting framework demonstrates a complete PoC workflow rather than an isolated prediction model. It connects data preparation, machine-learning inference, route-aware planning, supervisory control, safety validation, torque allocation, simulation, and KPI reporting. This provides a practical foundation for future Software-in-the-Loop, Hardware-in-the-Loop, and embedded real-time development under Project PHEX '26.

CHAPTER 3

CONCLUSION AND FUTURE SCOPE

3.1 Conclusion

The internship provided practical experience in the design and implementation of an intelligent Supervisory Energy Management System (EMS) for a Hybrid Electric Vehicle (HEV). The work involved developing predictive demand forecasting, route-aware State-of-Charge (SOC) planning, rule-based decision logic, safety override mechanisms, and torque-split control using Python. These modules were integrated into a unified simulation framework capable of selecting optimal operating modes under different driving conditions.

Simulation results demonstrated that the proposed EMS effectively balanced battery utilization, engine efficiency, regenerative braking, and overall energy consumption across multiple drive-cycle scenarios. The project also strengthened practical knowledge in machine learning, control systems, software architecture, data preprocessing, model evaluation, and modular software development. Overall, the internship provided valuable exposure to real-world automotive software development and intelligent vehicle energy management.

3.2 Future Scope

The developed PoC can be extended in several directions to support future phases of Project PHEX:

- Integration with real-time GPS navigation and live traffic information.
- Deployment of the EMS on embedded automotive hardware (ECU/SoC) for real-time execution.
- Implementation of SAE J1939 and CAN FD communication for interaction with vehicle subsystems [9, 10].
- Replacement of rule-based decision logic with Reinforcement Learning or Model Predictive Control (MPC).
- Validation using industry-standard drive cycles such as WLTP [11], NEDC, FTP-75, and real-world telemetry.
- Integration of battery degradation prediction and thermal management models.
- Hardware-in-the-Loop (HIL) and Software-in-the-Loop (SIL) testing before vehicle deployment.
- Development of a graphical driver interface for monitoring energy flow, battery SOC, and operating mode.

- Extension of the framework to Plug-in Hybrid Electric Vehicles (PHEVs) and Battery Electric Vehicles (BEVs).
- Optimization for production-grade embedded systems with low-latency inference and real-time safety monitoring.

APPENDICES

A.1 Offer Letter

Selection for Project PHEX '27 Internship - May 2026 External Inbox x



Martin Dsouza <martin.studentdirector.phex@gmail.com>
to me, dsouza.martin@btech.christuniversity.in, vanshi.sanjeev@psych.christuniversity.in

Thu, Apr 9, 6:10 PM

Dear Mishael,

Congratulations.

We are pleased to inform you that you have been **successfully selected** for the **Project PHEX '27 Internship, May 2026 Intake**.

We appreciate the **strong alignment** between your cover letter, technical background, and our project goals. Your managerial experience as a senior manager with AIESEC and your **confirmed ability to dedicate 7-10 hours weekly** to this internship reflect **excellent time management**. Furthermore, we value your **honest self-awareness**; we encourage you to overcome your shyness and continue asking questions so that you can make the **absolute most of this learning experience**.

What's Next:

You will receive an email containing your work profile next week. Following that, we will schedule a 1-on-1 Google Meet to deep dive into your technical skills and **thoroughly discuss your work profile** for the Project PHEX '27 Internship.

Welcome to the Internship!

Warm regards,
Martin Dsouza
Project Director & Technical Head

Vanshi S. Nair
Head of Non-Technicals and Operation

On behalf of the Project PHEX '27 Recruitment Team
SoET, CHRIST (Deemed to be University), Bangalore

A.2 Completion Certificate

	CHRIST (Deemed to be University) School of Engineering and Technology Kengeri, Bengaluru, Karnataka, India	
PROJECT PHEX '27 <i>Machine Learning & Edge Computing enabled Energy Management System for Optimization of Hybrid Electric Powertrain</i>		
CERTIFICATE OF COMPLETION		
This is to certify that		
MISHAEL JULIAN Reg. No. 2462184		
has successfully completed the internship under Project PHEX '27, contributing to the design and validation of a Proof-of-Concept Hybrid Electric Vehicle Energy Management System, encompassing predictive EMS concepts, machine-learning-assisted energy optimization frameworks, drive-cycle simulation, and technical documentation, during the period 04 May 2026 to 06 June 2026, in partial fulfilment of the requirements for the degree of Bachelor of Technology in Computer Science Engineering with Specialization in Artificial Intelligence and Machine Learning.		
 Mr. Martin Dsouza Student Project Technical Head Project PHEX '27	 Dr. Parag Jose Faculty Co-ordinator & Faculty Head COE E-mobility	 Dr. Pradeep Kumar G.S. Faculty Co-ordinator Co-ordinator Automotive Technology
Date Submitted: _____		

A.3 Code and Screenshots

A.3.1 Loading and Preparing Telemetry Data

Vehicle telemetry, driver behaviour, battery, and route datasets were loaded and transformed into a unified format before being used by the Energy Management System (EMS). The preprocessing pipeline removed duplicate records, handled missing values, detected and clipped statistical outliers, encoded categorical variables, standardized numerical features, and generated additional engineered features required for machine learning. Specialized preprocessing was also performed for IMU sensor measurements and NASA battery datasets to derive acceleration, gyroscope, battery health, and degradation features used by the predictive EMS controller.

Figure A.3.1 shows the preprocessing function applied to the driver-behaviour dataset. The routine removes duplicates, handles missing data and outliers, generates an efficiency target, encodes behaviour labels, and scales the selected numerical features.

```
def preprocess_behavior_dataset(df: pd.DataFrame) -> Tuple[pd.DataFrame, LabelEncoder, StandardScaler]:
    logger.info("\n" + "=" * 60)
    logger.info(" PREPROCESSING: Driver Behavior Dataset")
    logger.info("=" * 60)
    name = "Behavior"

    # 1. Duplicates
    df = remove_duplicates(df, name)

    # 2. Missing values
    df = handle_missing_values(df, name=name)

    # 3. Outlier detection
    numeric_cols = ["speed_kmph", "accel_x", "accel_y", "brake_pressure",
                    "steering_angle", "throttle", "lane_deviation",
                    "headway_distance", "reaction_time"]
    detect_outliers_iqr(df, columns=numeric_cols, name=name)

    # 4. Clip extreme outliers (only truly extreme ones, 3*IQR)
    df = clip_outliers(df, columns=numeric_cols, factor=3.0, name=name)

    # 5. Generate synthetic efficiency target
    df = generate_synthetic_efficiency(df)

    # 6. Encode behavior labels
    le = LabelEncoder()
    df["behavior_encoded"] = le.fit_transform(df["behavior_label"])
    logger.info(f" [{name}] Label encoding: {dict(zip(le.classes_, le.transform(le.classes_)))}")

    # 7. Scale numeric features
    feature_cols = numeric_cols.copy()
    scaler = StandardScaler()
    df_scaled = df.copy()
    df_scaled[feature_cols] = scaler.fit_transform(df[feature_cols])

    # Keep both scaled and unscaled versions
    # The unscaled version is useful for interpretability
    for col in feature_cols:
        df[f"{col}_scaled"] = df_scaled[col]
    logger.info(f" [{name}] Scaled {len(feature_cols)} numeric features (StandardScaler)")
    logger.info(f" [{name}] Final shape: {df.shape}")
    return df, le, scaler
```

Figure A.3.1: Driver Behaviour Dataset Preprocessing Pipeline

Figure A.3.2 illustrates the feature-engineering routine for raw IMU telemetry. Acceleration magnitude, gyroscope magnitude, axis-wise jerk, and total jerk are derived to represent vehicle motion and sudden driving inputs.

```
def engineer_imu_features(df: pd.DataFrame) -> pd.DataFrame:
    # Acceleration magnitude (total g-force)
    df["acc_magnitude"] = np.sqrt(df["AccX"]**2 + df["AccY"]**2 + df["AccZ"]**2)
    # Gyroscope magnitude (total angular velocity)
    df["gyro_magnitude"] = np.sqrt(df["GyroX"]**2 + df["GyroY"]**2 + df["GyroZ"]**2)
    # Jerk (rate of acceleration change) - computed as difference
    # High jerk = sudden driving inputs = aggressive behavior
    df["jerk_x"] = df["AccX"].diff().fillna(0)
    df["jerk_y"] = df["AccY"].diff().fillna(0)
    df["jerk_z"] = df["AccZ"].diff().fillna(0)
    df["jerk_magnitude"] = np.sqrt(df["jerk_x"]**2 + df["jerk_y"]**2 + df["jerk_z"]**2)
    logger.info(f" [Telemetry] Engineered features: acc_magnitude, gyro_magnitude, jerk_x/y/z, jerk_magnitude")
    return df
```

Figure A.3.2: IMU Telemetry Feature Engineering

Figure A.3.3 presents the conversion of NASA battery-cycle measurements into a tabular dataset. Voltage, current, temperature, duration, and capacity-proxy features are extracted for battery-health and degradation analysis.

```
def flatten_nasa_battery(
    battery_data: Dict[str, Any],
    max_steps: Optional[int] = None,
) -> pd.DataFrame:
    logger.info("\n" + "=" * 60)
    logger.info(" PREPROCESSING: NASA Battery Dataset")
    logger.info("=" * 60)
    rows = []
    for batt_name, mat_data in battery_data.items():
        steps = mat_data["data"]["step"]
        n_steps = len(steps) if max_steps is None else min(len(steps), max_steps)
        logger.info(f" [{batt_name}] Processing {n_steps:,} / {len(steps):,} steps...")
        for i in range(n_steps):
            step = steps[i]
            # Extract arrays - handle scalar edge cases
            voltage = np.atleast_1d(step["voltage"]).flatten()
            current = np.atleast_1d(step["current"]).flatten()
            temperature = np.atleast_1d(step["temperature"]).flatten()
            rel_time = np.atleast_1d(step["relativeTime"]).flatten()
            # Skip steps with insufficient data (< 3 data points)
            if len(voltage) < 3:
                continue
            # Compute aggregated features
            row = {
                "battery": batt_name,
                "step_index": i,
                "step_type": step["type"],           # C, D, or R
                "comment": step.get("comment", ""),
                # Voltage features
                "voltage_mean": voltage.mean(),
                "voltage_min": voltage.min(),
                "voltage_max": voltage.max(),
                "voltage_std": voltage.std(),
                "voltage_delta": voltage[-1] - voltage[0], # drop during step
                # Current features
                "current_mean": current.mean(),
                "current_min": current.min(),
                "current_max": current.max(),
                "current_std": current.std(),
                "current_abs_mean": np.abs(current).mean(),
                # Temperature features
                "temp_mean": temperature.mean(),
                "temp_min": temperature.min(),
                "temp_max": temperature.max(),
                "temp_delta": temperature[-1] - temperature[0],
                # Duration
                "duration_sec": rel_time[-1] if len(rel_time) > 0 else 0,
                "n_datapoints": len(voltage),
                # Derived: discharge capacity proxy
                "capacity": [mean_current] * duration (Ampere-seconds)
                "capacity_proxy": np.abs(current.mean()) * (rel_time[-1] if len(rel_time) > 0 else 0),
            }
            rows.append(row)
    df = pd.DataFrame(rows)
```

Figure A.3.3: NASA Battery Dataset Processing and Feature Extraction

Figure A.3.4 shows the generation of controller-ready EMS state variables. The routine creates power, SOC, road-context, regenerative-braking, thermal, and interaction features used by the predictive and supervisory modules.

```
def engineer_features(df: pd.DataFrame) -> pd.DataFrame:
    logger.info("Engineering EMS features...")
    # 1. Flags
    df["high_power_flag"] = (df["power_required_kw"] > HIGH_POWER_KW).astype(int)
    df["low_soc_flag"] = (df["battery_soc"] < SOC_LOW).astype(int)
    df["critical_soc_flag"] = (df["battery_soc"] < SOC_CRITICAL).astype(int)
    df["is_urban"] = (df["current_segment"] == 'urban').astype(int)
    df["is_highway"] = (df["current_segment"] == 'highway').astype(int)
    df["is_stop_go"] = (df["current_segment"] == 'stop_go').astype(int)
    df["regen_candidate"] = ((df["braking"] == True) &
                             (df["regen_available"] == True) &
                             (df["speed"] > REGEN_MIN_SPEED)).astype(int)
    df["next_is_highway"] = (df["next_segment"] == 'highway').astype(int)
    df["next_is_traffic"] = df["next_segment"].isin(['traffic', 'stop_go']).astype(int)
    # 2. Continuous features
    df["load_intensity_score"] = (df["power_required_kw"] + df["aux_load_kw"]) / 120.0
    # Thermal and grade
    df["thermal_stress_flag"] = ((df["battery_temp"] > BATTERY_TEMP_MAX) |
                                  (df["battery_temp"] < BATTERY_TEMP_MIN)).astype(int)
    df["uphill_flag"] = (df["grade_angle"] > 3.0).astype(int)
    df["downhill_flag"] = (df["grade_angle"] < -3.0).astype(int)
    # Ratios and interactions
    df["power_per_speed"] = df["power_required_kw"] / (df["speed"] + 1e-3)
    df["soc_x_power"] = df["battery_soc"] * df["power_required_kw"]
    logger.info(f"Engineered features added. New shape: {df.shape}")
    return df
```

Figure A.3.4: EMS Feature Engineering and State Variable Generation

A.3.2 Demand Forecasting Module

A machine learning-based demand forecasting module was developed to predict future vehicle power demand, speed, and acceleration using recent telemetry history. The module applies rolling-window temporal feature engineering together with Random Forest regression models to estimate future driving conditions. These predictions provide advisory information to the supervisory Energy Management System (EMS), enabling proactive operating mode selection and improved energy management performance.

This code initializes the `DemandForecaster` class and loads the trained Random Forest regression models for power demand, vehicle speed, and acceleration prediction. The trained models are stored locally and loaded dynamically during runtime.

```
class DemandForecaster:
    """
    Predicts future speed, acceleration, and power demand using rolling window history.
    """
    def __init__(self):
        self.power_model_path = MODELS_DIR / "power_forecaster.pkl"
        self.speed_model_path = MODELS_DIR / "speed_forecaster.pkl"
        self.accel_model_path = MODELS_DIR / "accel_forecaster.pkl"

        self.power_model = None
        self.speed_model = None
        self.accel_model = None
        self.load_models()

    def load_models(self) -> bool:
        """Loads trained regressor models from models/ directory."""
        try:
            if self.power_model_path.exists():
                self.power_model = joblib.load(self.power_model_path)
            if self.speed_model_path.exists():
                self.speed_model = joblib.load(self.speed_model_path)
            if self.accel_model_path.exists():
                self.accel_model = joblib.load(self.accel_model_path)

            logger.info("Successfully loaded all demand forecasting models.")
            return True
        except Exception as e:
            logger.error(f"Error loading demand forecasting models: {e}")
            return False
```

Figure A.3.5: Demand Forecasting Module Initialization and Model Loading

This section implements the core forecasting algorithm. Historical telemetry is transformed into lag-based temporal features, categorical variables are encoded, feature vectors are prepared, and the trained regression models predict future vehicle power demand, speed, and acceleration for the supervisory EMS.

```
def predict_future(self, df_history: pd.DataFrame) -> Tuple[float, float, float]:
    if self.power_model is None or self.speed_model is None or self.accel_model is None:
        # Try to load models dynamically if not loaded
        if not self.load_models():
            # Fallback to current values if models are missing
            last_row = df_history.iloc[-1]
            logger.warning("DemandForecaster models not trained. Using current state fallback.")
            return (float(last_row["power_required_kw"]),
                    float(last_row["speed"]),
                    float(last_row["acceleration"]))
    # Prepare the single row (last row of lagged df)
    # First build lags on the history df
    lagged_df = create_lags(df_history, ["speed", "acceleration", "power_required_kw"], n_lags=5)
    last_row_df = lagged_df.iloc[-1].copy()
    # Encode categorical columns
    if "traffic_condition" in last_row_df.columns:
        last_row_df["traffic_condition"] = last_row_df["traffic_condition"].map(TRAFFIC_MAP).fillna(0).astype(int)
    if "current_segment" in last_row_df.columns:
        last_row_df["current_segment"] = last_row_df["current_segment"].map(SEGMENT_MAP).fillna(0).astype(int)
    if "next_segment" in last_row_df.columns:
        last_row_df["next_segment"] = last_row_df["next_segment"].map(SEGMENT_MAP).fillna(0).astype(int)
    # Ensure boolean columns are numeric
    for col in ["regen_available", "braking"]:
        if col in last_row_df.columns:
            last_row_df[col] = last_row_df[col].astype(int)
    # Features used for training (must match exactly)
    feature_cols = [
        "speed", "acceleration", "power_required_kw", "battery_soc", "battery_temp",
        "aux_load_kw", "grade_angle", "regen_available", "braking",
        "traffic_condition", "current_segment", "next_segment",
        "speed_lag_1", "speed_lag_2", "speed_lag_3", "speed_lag_4", "speed_lag_5",
        "acceleration_lag_1", "acceleration_lag_2", "acceleration_lag_3", "acceleration_lag_4", "acceleration_lag_5",
        "power_required_kw_lag_1", "power_required_kw_lag_2", "power_required_kw_lag_3", "power_required_kw_lag_4", "power_required_kw_lag_5"
    ]
    # Ensure all columns exist, if not fill with 0.0
    for col in feature_cols:
        if col not in last_row_df.columns:
            last_row_df[col] = 0.0
    # Align features
    X = last_row_df[feature_cols]
    pred_power = float(self.power_model.predict(X)[0])
    pred_speed = float(self.speed_model.predict(X)[0])
    pred_accel = float(self.accel_model.predict(X)[0])
    # INTEGRATION POINT - Alex: replace with real power_required_kw or transient load models
    return pred_power, pred_speed, pred_accel
```

Figure A.3.6: Future Vehicle Demand Prediction Algorithm

This code trains three Random Forest regression models using trip-level data partitioning to prevent temporal data leakage. The implementation generates lag features, constructs future prediction targets, evaluates model performance using MAE, RMSE, and R^2 metrics, and stores the trained models for deployment within the predictive EMS.

```
def train_demand_forecaster(data_path: Path, model_dir: Path, n_lags: int = 5, forecast_step: int = 5) -> Dict[str, Any]:
    logger.info(f"Loading data from {data_path} to train demand forecaster...")
    df = pd.read_csv(data_path)

    # 1. Generate lagged features within trip groups to prevent leakage
    df_features = create_lags(df, ["speed", "acceleration", "power_required_kw"], n_lags=n_lags)

    # 2. Map categorical columns
    df_features["traffic_condition"] = df_features["traffic_condition"].map(TRAFFIC_MAP).fillna(0).astype(int)
    df_features["current_segment"] = df_features["current_segment"].map(SEGMENT_MAP).fillna(0).astype(int)
    df_features["next_segment"] = df_features["next_segment"].map(SEGMENT_MAP).fillna(0).astype(int)

    # Ensure boolean columns are numeric
    for col in ["regen_available", "braking"]:
        if col in df_features.columns:
            df_features[col] = df_features[col].astype(int)

    # 3. Define feature columns
    feature_cols = [
        "speed", "acceleration", "power_required_kw", "battery_soc", "battery_temp",
        "aux_load_kw", "grade_angle", "regen_available", "braking",
        "traffic_condition", "current_segment", "next_segment",
        "speed_lag_1", "speed_lag_2", "speed_lag_3", "speed_lag_4", "speed_lag_5",
        "acceleration_lag_1", "acceleration_lag_2", "acceleration_lag_3", "acceleration_lag_4", "acceleration_lag_5",
        "power_required_kw_lag_1", "power_required_kw_lag_2", "power_required_kw_lag_3", "power_required_kw_lag_4", "power_required_kw_lag_5"
    ]

    # 4. Generate future targets within each trip_id group to prevent lookahead leak across trips
    grouped = df_features.groupby("trip_id")
    df_features["target_power"] = grouped["power_required_kw"].shift(-forecast_step)
    df_features["target_speed"] = grouped["speed"].shift(-forecast_step)
    df_features["target_accel"] = grouped["acceleration"].shift(-forecast_step)

    # Drop rows where target is NaN (which occur at the end of each trip due to shift)
    df_clean = df_features.dropna(subset=["target_power", "target_speed", "target_accel"]).copy()
    X = df_clean[feature_cols]
    y_power = df_clean["target_power"]
    y_speed = df_clean["target_speed"]
    y_accel = df_clean["target_accel"]

    # Perform a trip-level split or randomized row split (trip-level split is more robust for sequences)
    # Since each trip has a distinct trip id, we can split trips:
    unique_trips = df_clean["trip_id"].unique()
    np.random.seed(RANDOM_STATE)
    np.random.shuffle(unique_trips)
    split_idx = int(len(unique_trips) * (1.0 - TEST_SIZE))
    train_trips = unique_trips[:split_idx]
    test_trips = unique_trips[split_idx:]
    train_mask = df_clean["trip_id"].isin(train_trips)
    test_mask = df_clean["trip_id"].isin(test_trips)
    X_train, X_test = X[train_mask], X[test_mask]
    y_power_train, y_power_test = y_power[train_mask], y_power[test_mask]
    y_speed_train, y_speed_test = y_speed[train_mask], y_speed[test_mask]
    y_accel_train, y_accel_test = y_accel[train_mask], y_accel[test_mask]
    logger.info(f"Split data into {len(train_trips)} train trips and {len(test_trips)} test trips.")
    logger.info(f"Train shape: {X_train.shape}, Test shape: {X_test.shape}")

    # 5. Train Random Forests (lightweight for fast training & stable inference)
    # Using small max_depth to guarantee generalization and avoid overfitting
    model_params = {"n_estimators": 50, "max_depth": 8, "random_state": RANDOM_STATE, "n_jobs": -1}
    logger.info("Training power demand forecaster...")
    power_rf = RandomForestRegressor(**model_params)
    power_rf.fit(X_train, y_power_train)
    pred_power = power_rf.predict(X_test)
    mae_power = mean_absolute_error(y_power_test, pred_power)
    rmse_power = np.sqrt(mean_squared_error(y_power_test, pred_power))
    r2_power = r2_score(y_power_test, pred_power)
    logger.info(f"Power Forecaster: MAE={mae_power:.3f} kW, RMSE={rmse_power:.3f} kW, R2={r2_power:.3f}")

    logger.info("Training speed forecaster...")
    speed_rf = RandomForestRegressor(**model_params)
    speed_rf.fit(X_train, y_speed_train)
    pred_speed = speed_rf.predict(X_test)
    mae_speed = mean_absolute_error(y_speed_test, pred_speed)
    rmse_speed = np.sqrt(mean_squared_error(y_speed_test, pred_speed))
    r2_speed = r2_score(y_speed_test, pred_speed)
    logger.info(f"Speed Forecaster: MAE={mae_speed:.3f} km/h, RMSE={rmse_speed:.3f} km/h, R2={r2_speed:.3f}")

    logger.info("Training acceleration forecaster...")
    accel_rf = RandomForestRegressor(**model_params)
    accel_rf.fit(X_train, y_accel_train)
    pred_accel = accel_rf.predict(X_test)
    mae_accel = mean_absolute_error(y_accel_test, pred_accel)
    rmse_accel = np.sqrt(mean_squared_error(y_accel_test, pred_accel))
    r2_accel = r2_score(y_accel_test, pred_accel)
    logger.info(f"Acceleration Forecaster: MAE={mae_accel:.3f} m/s², RMSE={rmse_accel:.3f} m/s², R2={r2_accel:.3f}")
```

Figure A.3.7: Random Forest Demand Forecaster Training Pipeline

A.3.3 Hybrid Decision Engine

The Hybrid Decision Engine integrates machine learning predictions with deterministic supervisory rules and safety validation to select an appropriate vehicle operating mode.

Figure A.3.8 initializes the Hybrid Decision Engine by loading the trained ML models, rule engine, safety layer, and telemetry history buffer. It also restores the feature definition required to keep runtime inputs consistent with model training.

```
class HybridDecisionEngine:
    def __init__(self, model_name: str = "ems_best_model.pkl"):
        self.rule_engine = RuleBasedEMS()
        self.predictive_controller = PredictiveEMSController()
        self.safety_layer = SafetyOverrideLayer()
        self.demand_forecaster = DemandForecaster()
        # Load ML Assets
        model_path = MODELS_DIR / model_name
        encoder_path = MODELS_DIR / "ems_label_encoder.pkl"
        if not model_path.exists() or not encoder_path.exists():
            raise FileNotFoundError(
                f"Missing ML models in {MODELS_DIR}. Run run_ems.py --train first."
            )
        self.model = joblib.load(model_path)
        self.label_encoder = joblib.load(encoder_path)
        # Load expected feature names (saved during training)
        feature_path = MODELS_DIR / "ems_feature_names.pkl"
        if feature_path.exists():
            self.expected_features = joblib.load(feature_path)
        else:
            self.expected_features = None
        # State history for online temporal feature engineering & demand forecasting
        self.state_history = deque(maxlen=15)
    def _update_history(self, state: VehicleState):
        """Append state to history, padding with duplicates if history is empty."""
        if len(self.state_history) == 0:
            for _ in range(15):
                self.state_history.append(state)
        else:
            self.state_history.append(state)
    def _get_history_dataframe(self) -> pd.DataFrame:
        """Convert deque of VehicleState to DataFrame."""
        rows = []
        for s in self.state_history:
            row_dict = {k: v for k, v in s.__dict__.items() if k != "timestamp"}
            rows.append(row_dict)
        return pd.DataFrame(rows)
    def _prepare_features(self, df_history: pd.DataFrame,
                        advisory: Optional[PredictiveAdvisory] = None) -> pd.DataFrame:
        """Apply the exact feature engineering used in training."""
```

Figure A.3.8: Hybrid Decision Engine Initialization and Model Loading

Figure A.3.9 presents the ML-based EMS workflow, including history updates, feature preparation, operating-mode prediction, confidence estimation, and safety validation. Unsafe predictions are rejected in favour of the deterministic rule-engine result.

```
def decide(self, state: VehicleState,
          lookahead: list = None,
          distance_remaining: float = 10.0,
          grade_ahead: float = 0.0) -> EMSDecision:
    # 1. Update history
    self._update_history(state)
    # 2. Predictive Advisory
    if lookahead is None:
        lookahead = [state.next_segment]
    advisory = self.predictive_controller.advise(
        state, lookahead, distance_remaining, grade_ahead
    )
    # 3. Prepare ML Input from history
    df_history = self._get_history_dataframe()
    # Suppress logging for row-by-row simulation
    prep_logger = logging.getLogger("src.preprocessing")
    prev_level = prep_logger.getEffectiveLevel()
    prep_logger.setLevel(logging.WARNING)
    X = self._prepare_features(df_history, advisory)
    prep_logger.setLevel(prev_level)
    # 4. ML Prediction
    pred_idx = self.model.predict(X)[0]
    ml_mode = self.label_encoder.inverse_transform([pred_idx])[0]
    if hasattr(self.model, "predict_proba"):
        probs = self.model.predict_proba(X)[0]
        confidence = float(probs[pred_idx])
    else:
        confidence = 1.0
    # 5. Safety Override Check using SafetyOverrideLayer
    is_safe, violation = self.safety_layer.check_safety(ml_mode, state)
    if is_safe:
        # Build advisory reason string
        adv_notes = []
        if advisory.preserve_battery:
            adv_notes.append("preserve_battery")
        if advisory.prepare_regen:
            adv_notes.append("prepare_regen")
        if advisory.prefer_engine:
            adv_notes.append("prefer_engine")
        if advisory.charge_sustain:
            adv_notes.append("charge_sustain")
        adv_str = f" [Advisory: {' ', ' '.join(adv_notes)}]" if adv_notes else ""
        return EMSDecision(
            mode=ml_mode,
            confidence=confidence,
            rule_triggered="ML_PRIMARY",
            reason=(f"ML selected {ml_mode} ({confidence*100:.1f}% conf).{adv_str}"),
            source="ML",
            input_state=state
        )
    else:
        # 6. Fallback to Deterministic Rules
        logger.debug(f"SAFETY OVERRIDE: {violation}")
        rule_decision = self.rule_engine.decide(state)
        rule_decision.source = "HYBRID_OVERRIDE"
        rule_decision.reason = (f"OVERRIDE: {violation} | "
                               f"Fallback -> {rule_decision.reason}")
        return rule_decision
```

Figure A.3.9: Machine Learning-Based EMS Decision-Making Workflow

Figure A.3.10 presents the rule-based EMS controller that selects the operating mode using vehicle, battery, traffic, and route conditions. The ordered rules prioritize regenerative braking, battery protection, traffic awareness, route preview, and efficient engine operation.

```
def select_mode(
    telemetry: dict,
    traffic_state: TrafficState,
    soc_plan: SOCPlan,
    battery_health: BatteryHealthState,
    predicted_demand_kw: float,
    regen_result: RegenResult,
) -> EMSMode:
    speed = telemetry.get("speed_kmh", 0.0)
    soc = telemetry.get("soc", 0.0)
    soc_frac = soc / 100.0 if soc > 1.0 else soc
    power_demand = telemetry.get("power_demand_kw", 0.0)
    throttle = telemetry.get("throttle", 0.0)
    current_segment = telemetry.get("current_segment", "urban")
    next_segment = telemetry.get("next_segment", "urban")
    position = telemetry.get("position_kn", 0.0)
    rpm = telemetry.get("rpm", 0.0)

    # 1. Regenerative Braking
    if power_demand < 0.0:
        if soc_frac >= MAX_SOC:
            logger.info("RULE6_REGEN: Regen disabled - battery full (SOC=%2f)", soc_frac)
        else:
            clamped_regen = min(abs(power_demand), MOTOR_PEAK_POWER)
            # 1 Wh = 1/3600 kWh
            recovered_wh = clamped_regen * (1.0 / 3600.0) * 1000.0
            logger.info("RULE6_REGEN: Regenerative braking active. Recovered Wh: %4f", recovered_wh)
            return EMSMode.REGEN

    # 2. Maximum Acceleration
    if throttle >= 0.90:
        logger.info("RULE7_MAX_ACCEL: Maximum acceleration requested.")
        return EMSMode.HYBRID_ASSIST

    # 3. Low SOC Recovery
    if soc_frac <= 0.25:
        logger.info("RULE8_SOC_RECOVERY: Low SOC (SOC=%2f) recovery active.", soc_frac)
        return EMSMode.CHARGE_SUSTAIN

    # 4. Urban Stop-and-Go EV Priority
    if speed < 50.0 and (traffic_state == TrafficState.CONGESTED or traffic_state.value == "CONGESTED") and current_segment.lower() == "urban":
        if soc_frac <= (MIN_SOC + 0.05):
            logger.info("RULE1_SUPPRESSED_LOW_SOC: Urban EV Priority suppressed due to low SOC (SOC=%2f)", soc_frac)
        else:
            logger.info("RULE1_EV_PRIORITY: Urban stop-and-go prioritising EV operation.")
            return EMSMode.EV

    # 5. SOC Preservation Before Urban Zones
    upcoming_urban = next_segment.lower() in ("urban", "stop_go")
    if upcoming_urban and soc_frac < 0.80 and current_segment.lower() in ("highway", "mixed", "arterial"):
        logger.info("RULE2_SOC_PRESERVE: Upcoming urban zone - preserving SOC (SOC=%2f).", soc_frac)
        return EMSMode.CHARGE_SUSTAIN

    # 6. Traffic-Aware Mode Planning
    if traffic_state == TrafficState.CONGESTED or traffic_state.value == "CONGESTED":
        logger.info("RULE3_TRAFFIC_AWARE: High congestion forecast - biasing EV/Hybrid mode.")
        return EMSMode.HYBRID_ASSIST

    # 7. GPS Route Preview Mode Planning
    if current_segment.lower() == "urban":
        logger.info("RULE4_ROUTE_PREVIEW: Urban segment - favouring EV mode.")
        return EMSMode.EV
    elif current_segment.lower() == "highway":
        logger.info("RULE4_ROUTE_PREVIEW: Highway segment - favouring ICE efficiency mode.")
        return EMSMode.ICE

    # 8. ICE Efficiency Sweet Spot (C.RULE5)
    if rpm > 0.0 and (rpm < 2000.0 or rpm > 3000.0):
        logger.info("RULE5_RPM_SWEET_SPOT: Engine outside peak band (%d RPM). Adjusting split.", int(rpm))
        if rpm > 3000.0:
            # Shed load
            return EMSMode.HYBRID_ASSIST
        else:
            # Deactivate or increase load
            return EMSMode.EV

    # Default fallback
    if soc_frac > MIN_SOC:
        return EMSMode.EV
    else:
        return EMSMode.CHARGE_SUSTAIN
```

Figure A.3.10: Supervisory Rule-Based Energy Management Decision Logic

REFERENCES

- [1] A. Sciarretta and L. Guzzella, “Control of hybrid electric vehicles,” *IEEE Control Systems Magazine*, vol. 27, no. 2, pp. 60–70, Apr. 2007.
- [2] C. C. Chan, “The state of the art of electric, hybrid, and fuel cell vehicles,” *Proceedings of the IEEE*, vol. 95, no. 4, pp. 704–718, Apr. 2007.
- [3] I. Husain, *Electric and Hybrid Vehicles: Design Fundamentals*, 3rd ed. Boca Raton, FL, USA: CRC Press, 2021.
- [4] J. Larminie and J. Lowry, *Electric Vehicle Technology Explained*, 2nd ed. Chichester, U.K.: Wiley, 2012.
- [5] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794.
- [6] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [7] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [8] Mercedes-Benz Group AG, *Mercedes-Benz A250e Plug-In Hybrid Technical Specifications*, Stuttgart, Germany, 2024.
- [9] SAE International, *SAE J1939 Digital Annex for Commercial Vehicle Communication Networks*, Warrendale, PA, USA, 2024.
- [10] Bosch GmbH, *CAN FD (Controller Area Network Flexible Data Rate) Specification Version 1.0*, Stuttgart, Germany, 2012.
- [11] WLTP, *Worldwide Harmonized Light Vehicles Test Procedure (WLTP)*, United Nations Economic Commission for Europe (UNECE), Geneva, Switzerland, 2023.